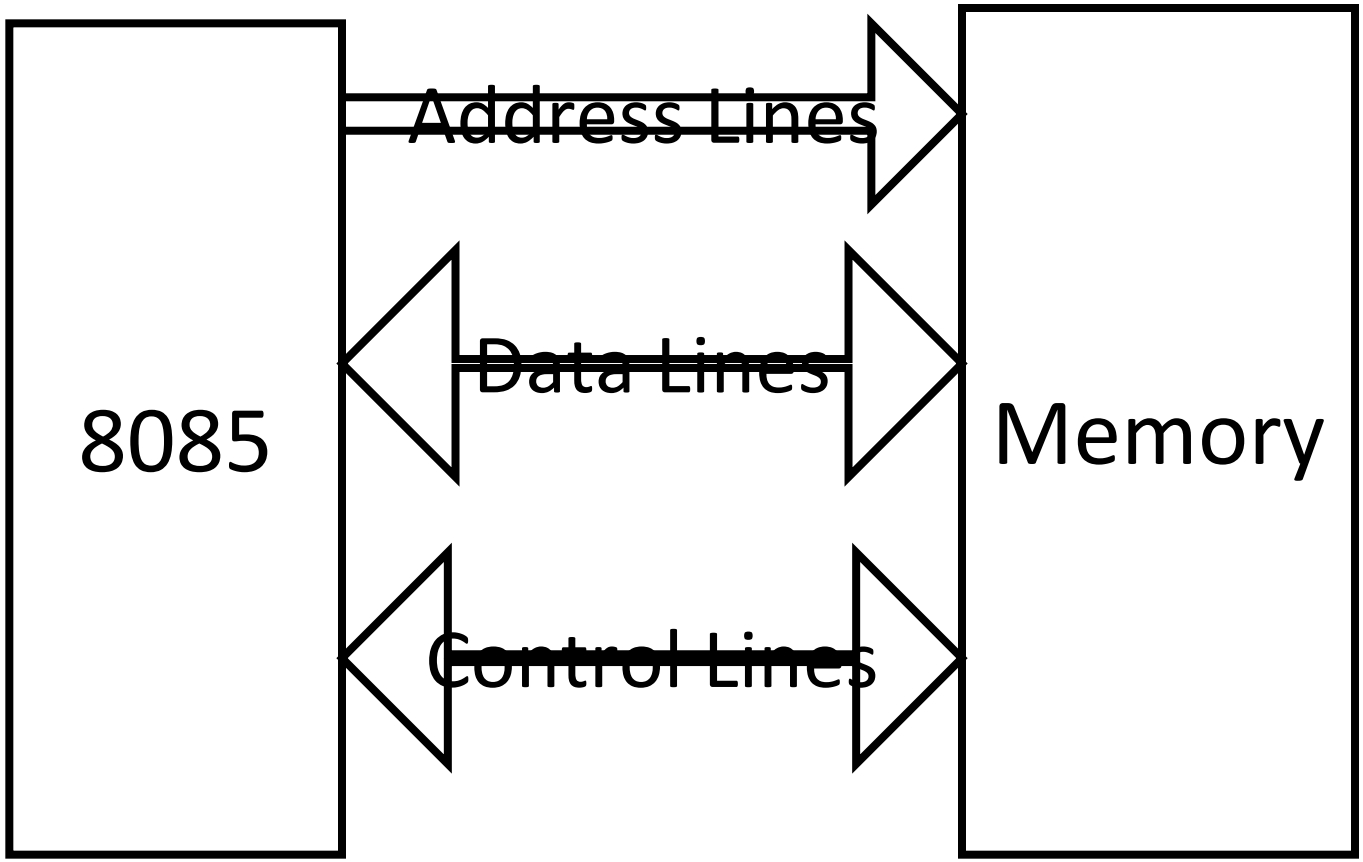
What is an Interface

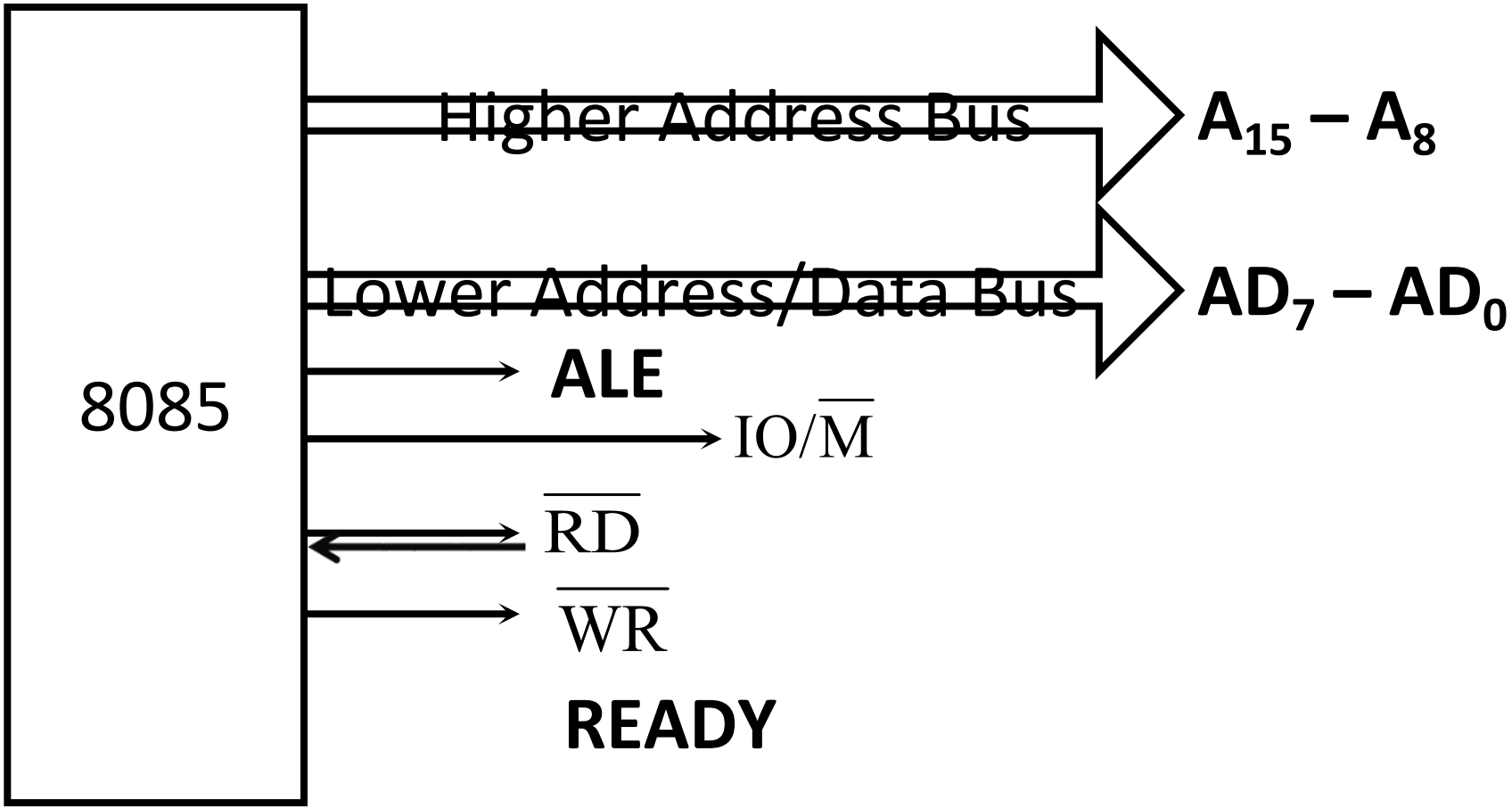
- An **interface** is a concept that refers to a point of interaction between components, and is applicable at the level of both hardware and software.
- This allows a component, (such as a **graphics card** or an **Internet browser**), to function independently while using interfaces to communicate with other components via an **input/output system** and an **associated protocol**.

Memory and I/O Interfacing

Example Block Diagram

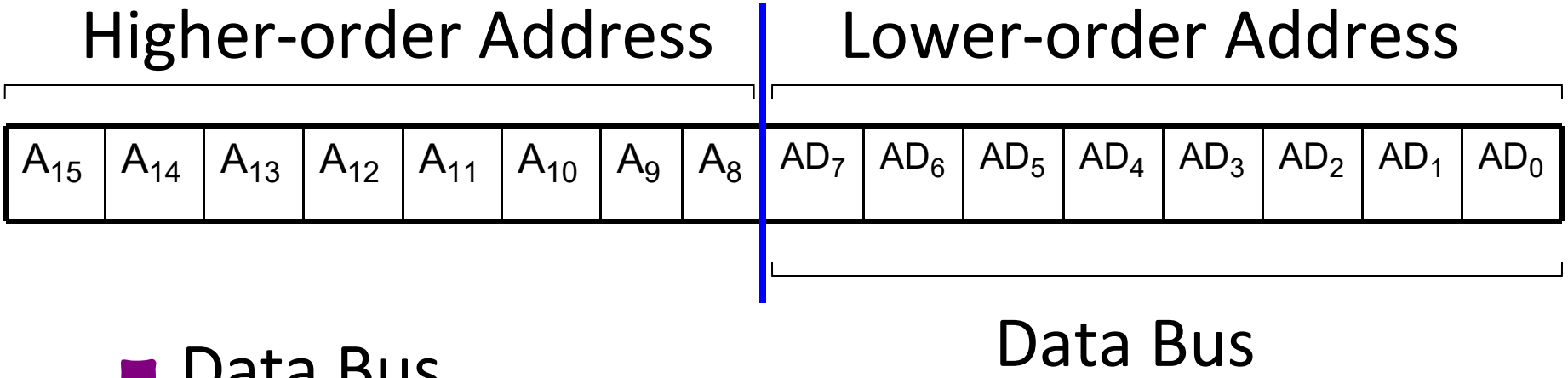


8085 Interfacing Pins



Address Bus of 8085

- Address Bus
  - Used to address memory & I/O devices
  - 8085 has a 16-bit address bus



- Data Bus
  - Used to transfer instructions and data
  - 8085 has a 8-bit data bus

Higher Order Address Bus

- The higher order address bus is a unidirectional bus.
- It carries most significant 8-bits of a 16-bit address of memory or I/O device.
- Address remains on lines as long operation is not completed.

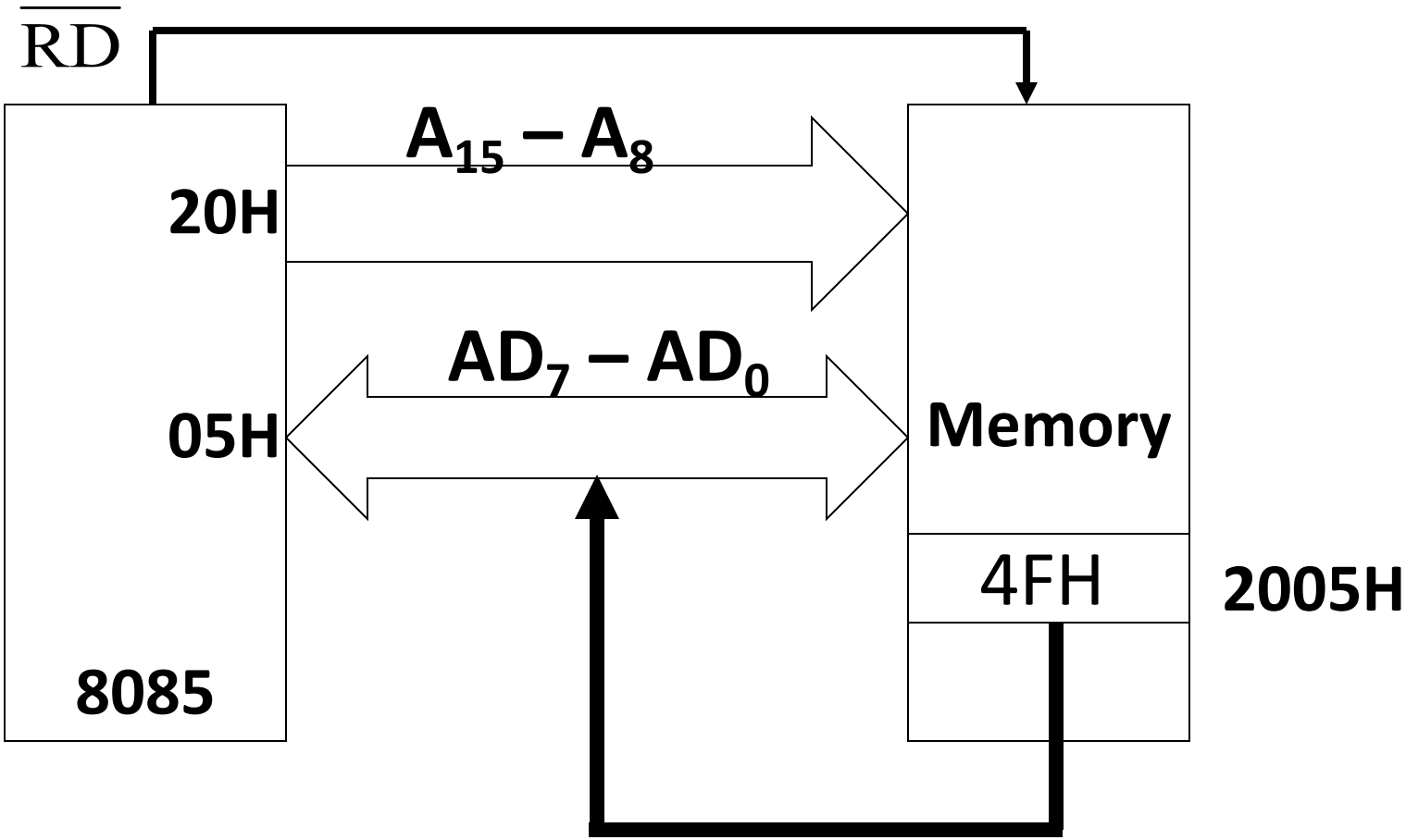
Lower Order Address/Data Bus

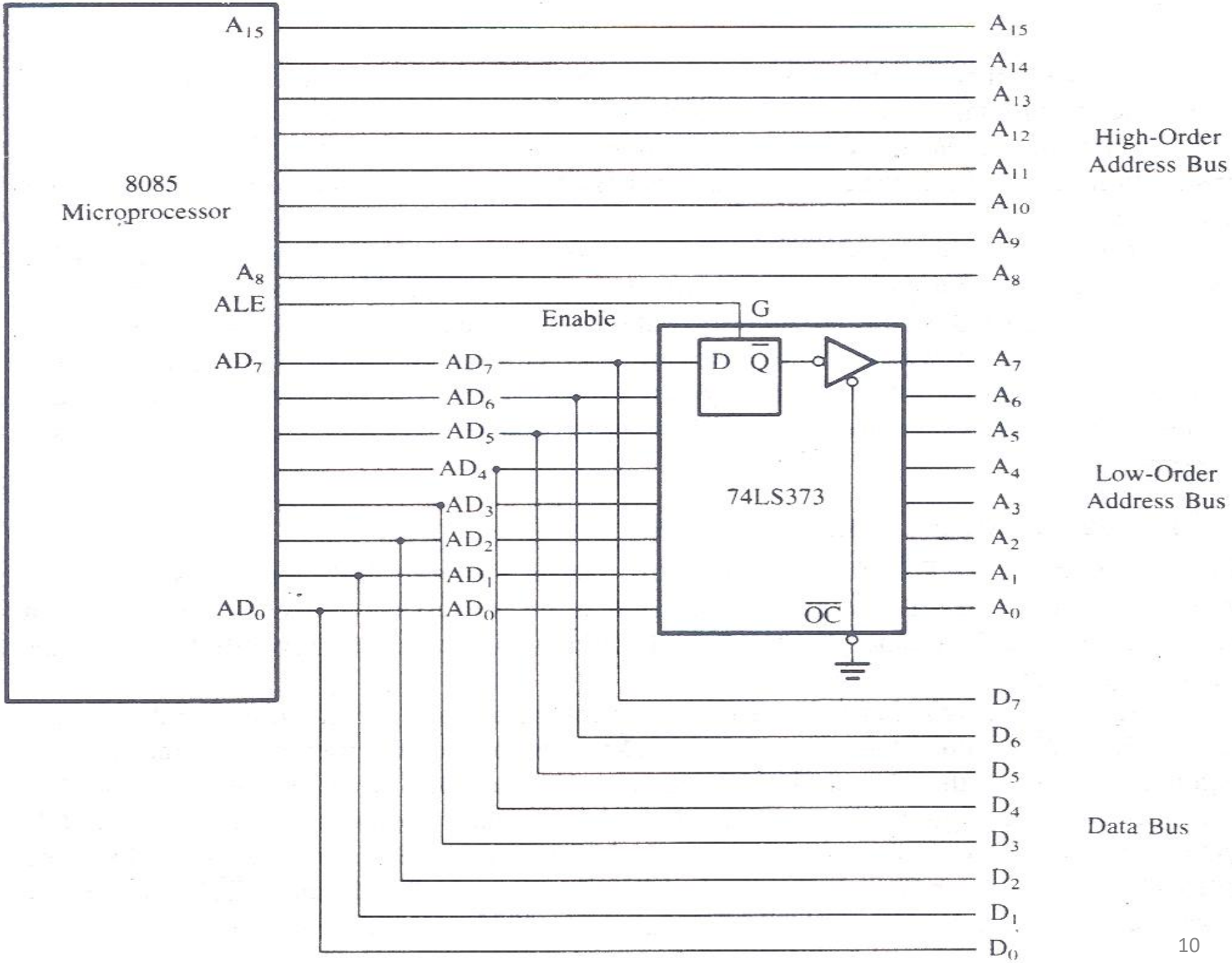
- This bus is bidirectional and works on time division multiplexing between address and data.
- During first clock cycle, it serves as a least significant 8-bits of memory/ IO address.
- For second and third clock cycles it acts as data bus and carries data.

Demultiplexing Address/Data Lines

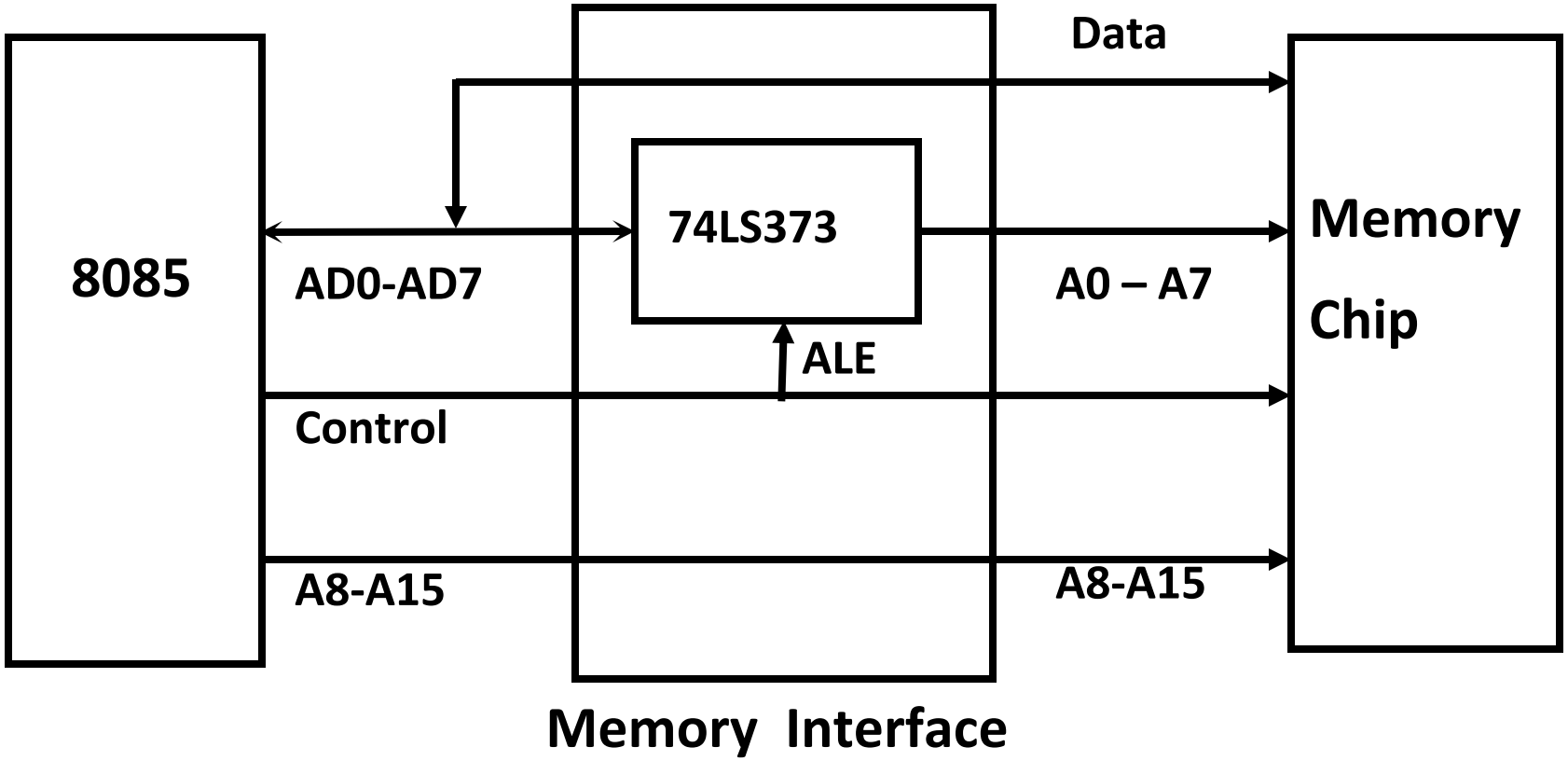
- 8085 identifies a memory location with its 16 address lines, (AD0 to AD7) & (A8 to A15)
- 8085 performs data transfer using its data lines, AD0 to AD7
- Lower order address bus & Data bus are multiplexed on same lines i.e. AD0 to AD7.
- Demultiplexing refers to separating Address & Data signals for read/write operations.

Need for Demultiplexing…

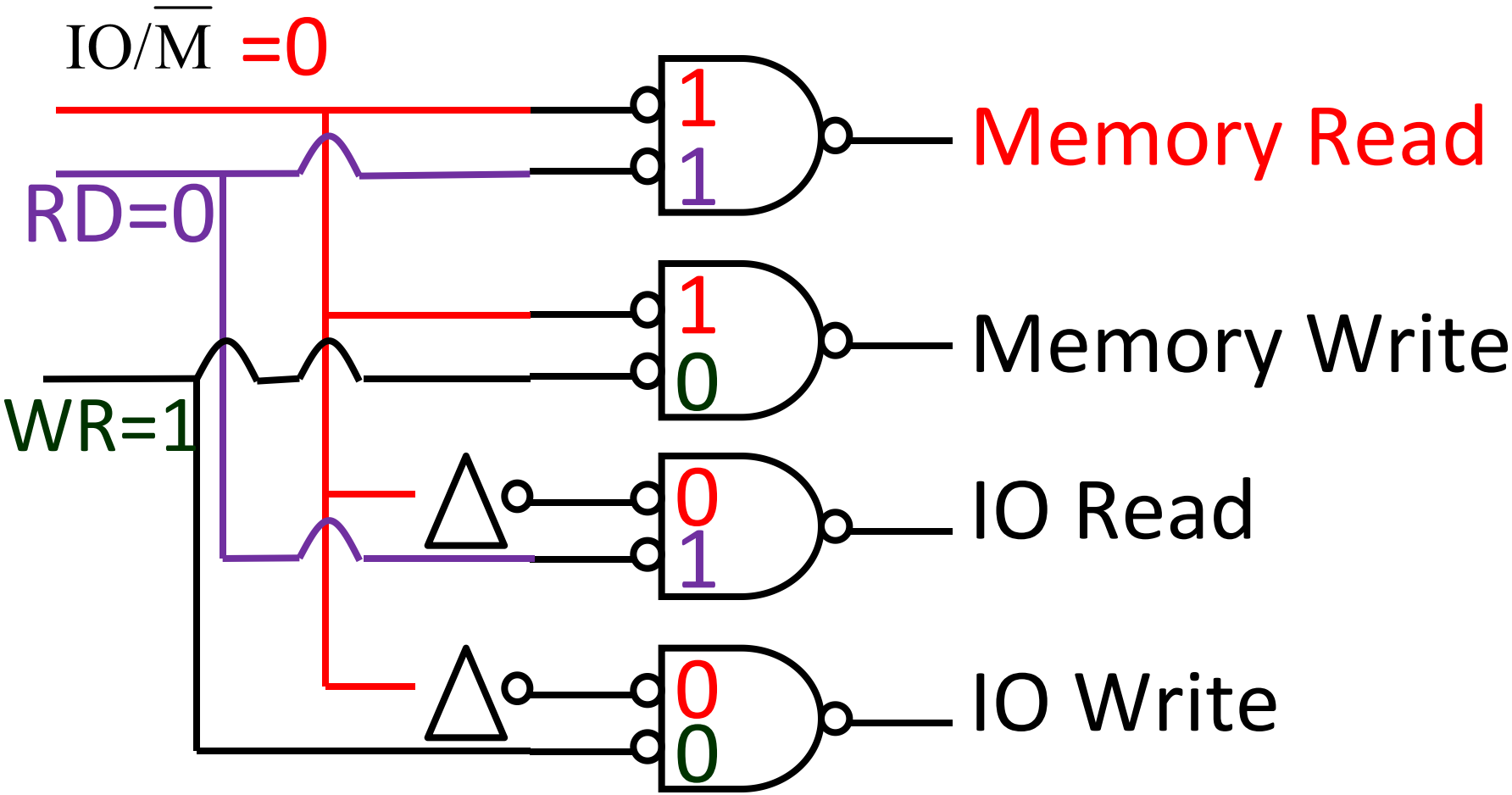



8085
Microprocessor
A15
A8
ALE
AD7
AD0
Enable
G
D
Q
74LS373
OC
AD7
AD6
AD5
AD4
AD3
AD2
AD1
AD0
A15
A14
A13
A12
A11
A10
A9
A8
A7
A6
A5
A4
A3
A2
A1
A0
D7
D6
D5
D4
D3
D2
D1
D0
High-Order
Address Bus
Low-Order
Address Bus
Data Bus

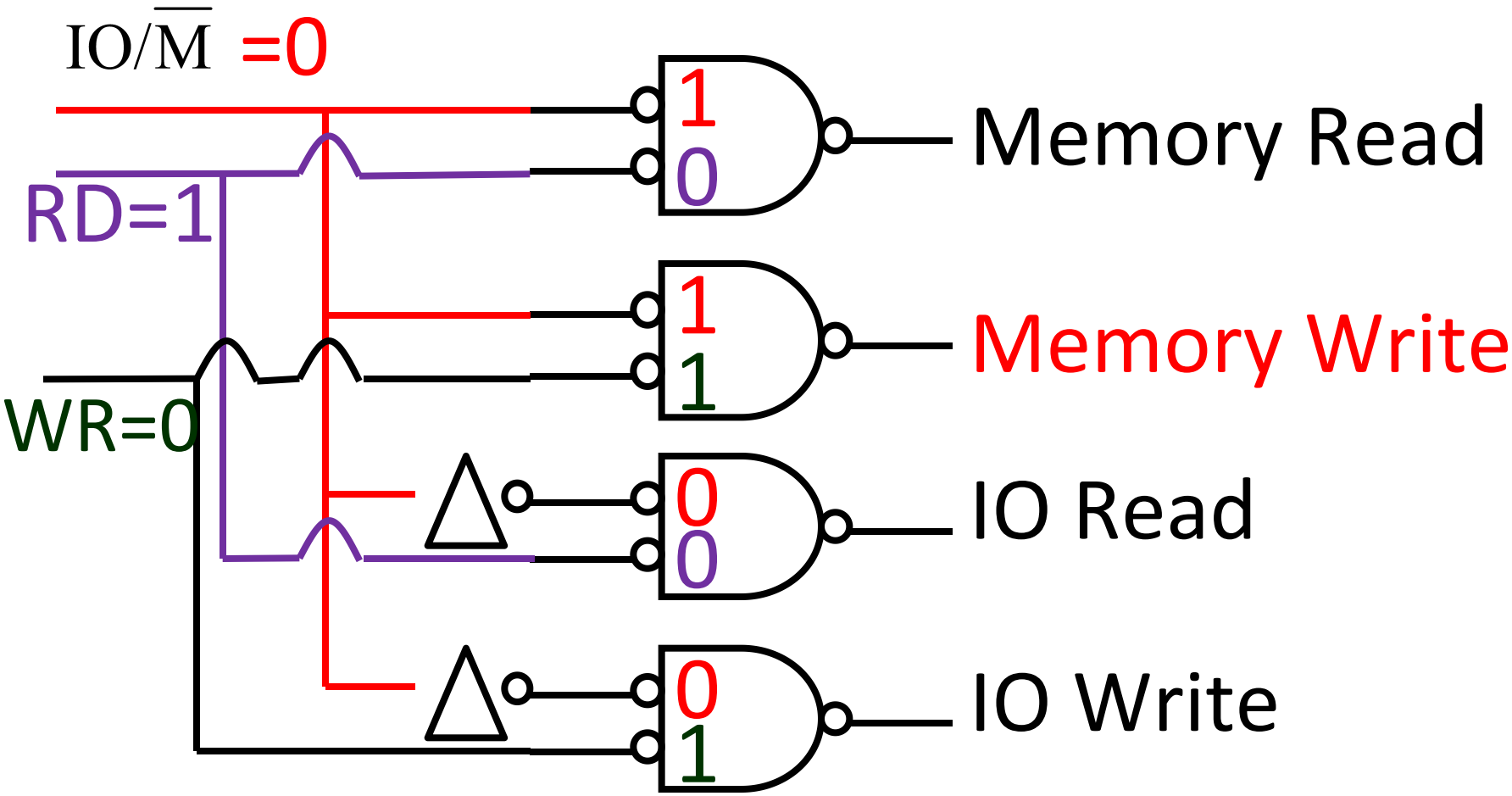
Demultiplexing Address/Data Lines



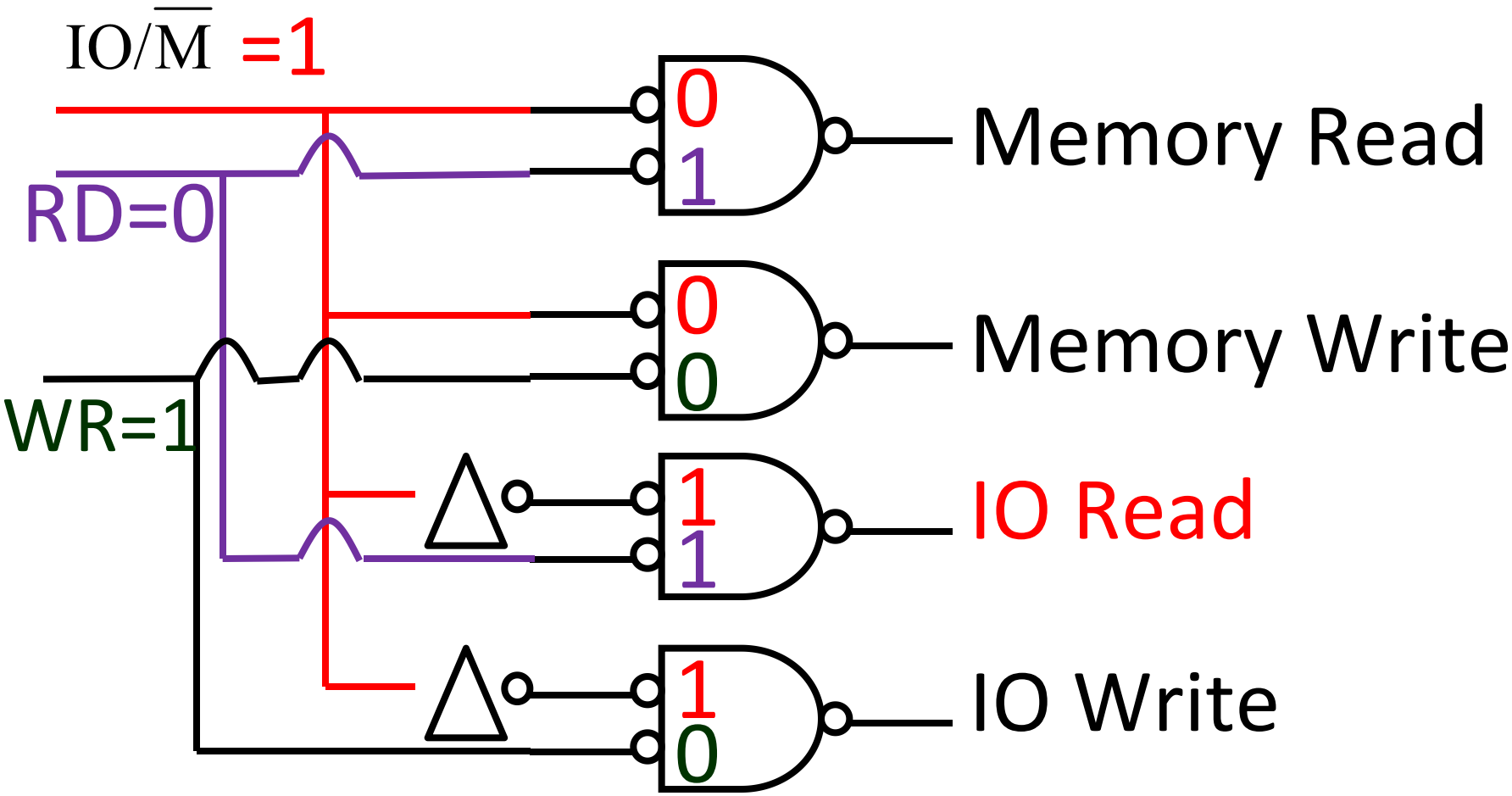
Generating Control Signals



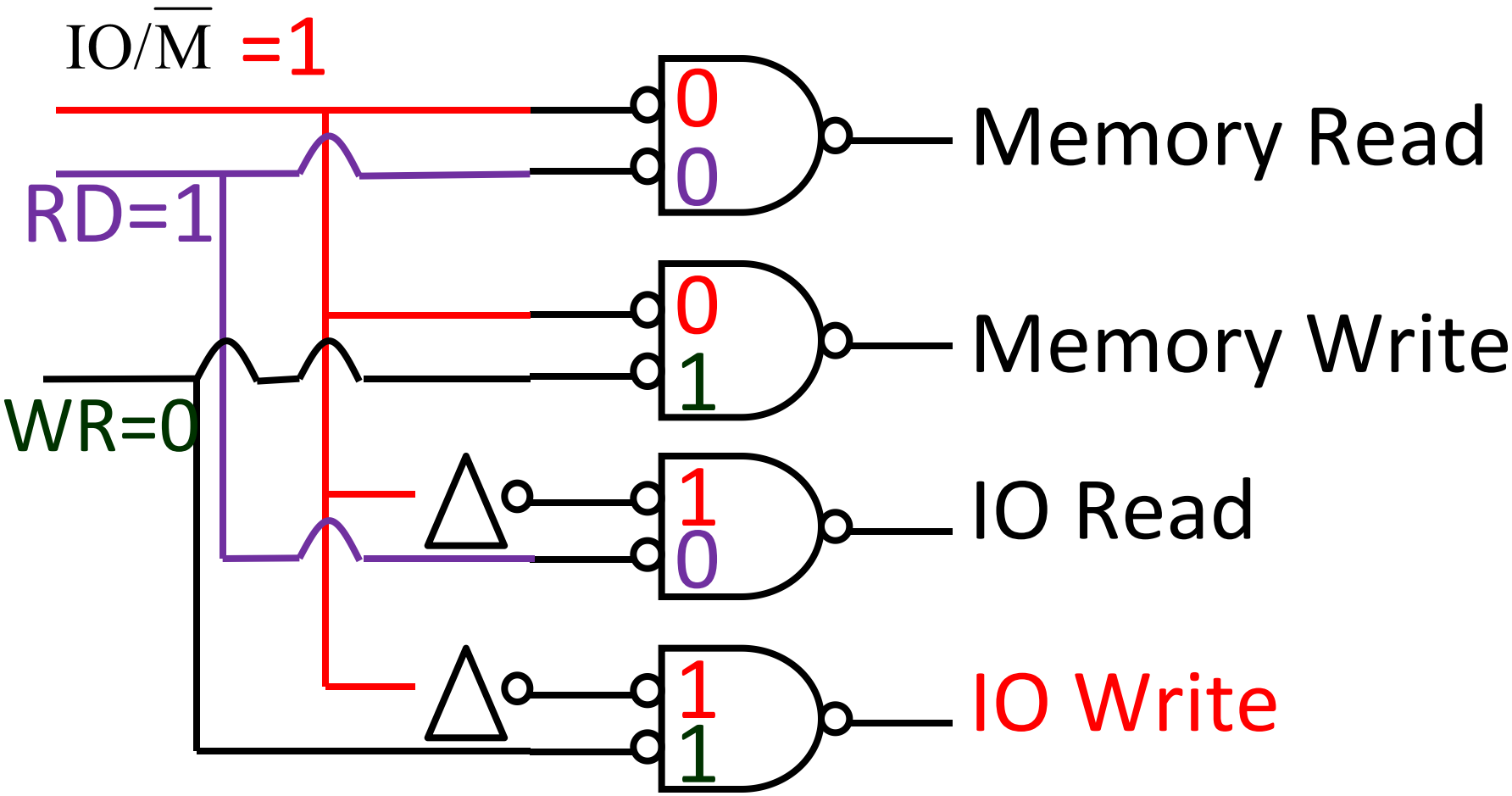
Generating Control Signals



Generating Control Signals



Generating Control Signals



Memory Interface

- **The memory is made up of semiconductor material used to store the programs and data. The types of memory is,**
  - **Primary or main memory**
  - **Secondary memory**

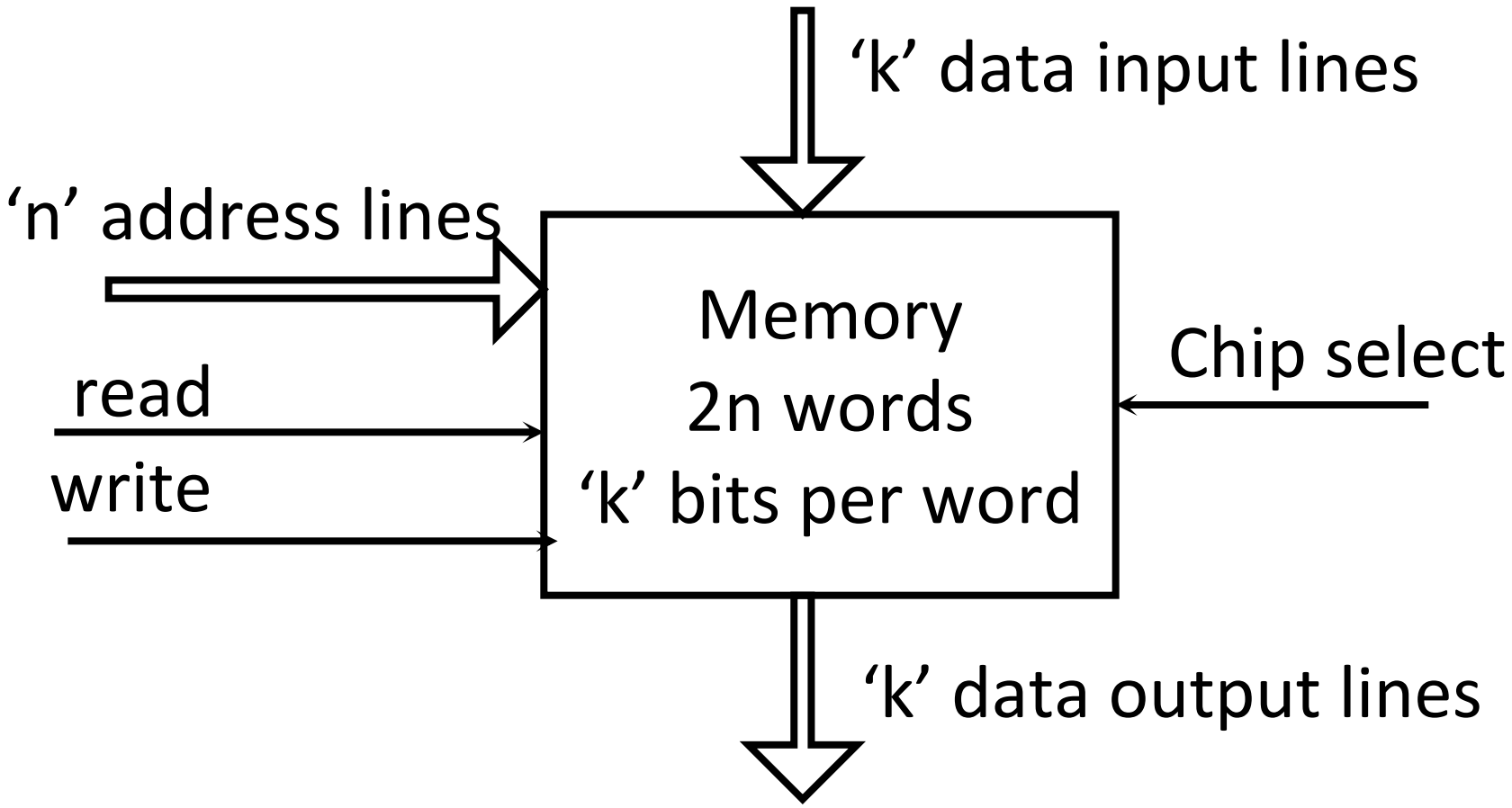
Primary Memory

- RAM and ROM are examples of this type of memory.
- Microprocessor uses it in storing a program temporarily (commonly called loading) and executing a program.
- Hence the speed of this type of memory should be fast.

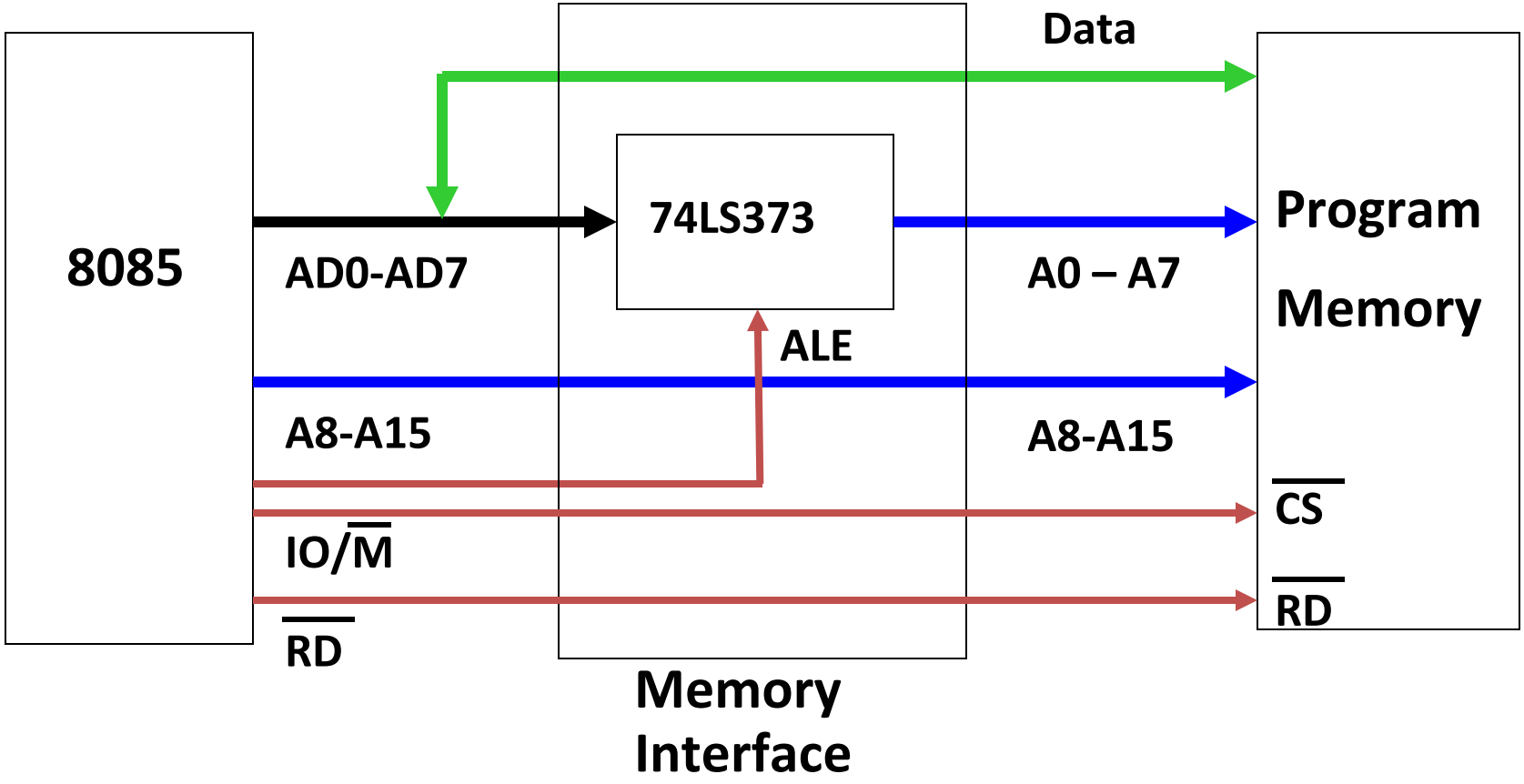
Secondary Memory

- These are used for bulk storage of data and information.
- The main examples include Floppy, Hard Disk, CD-ROM, Magnetic Tape etc.
- Slower and Sequential Access Nature.
- non-volatile nature.

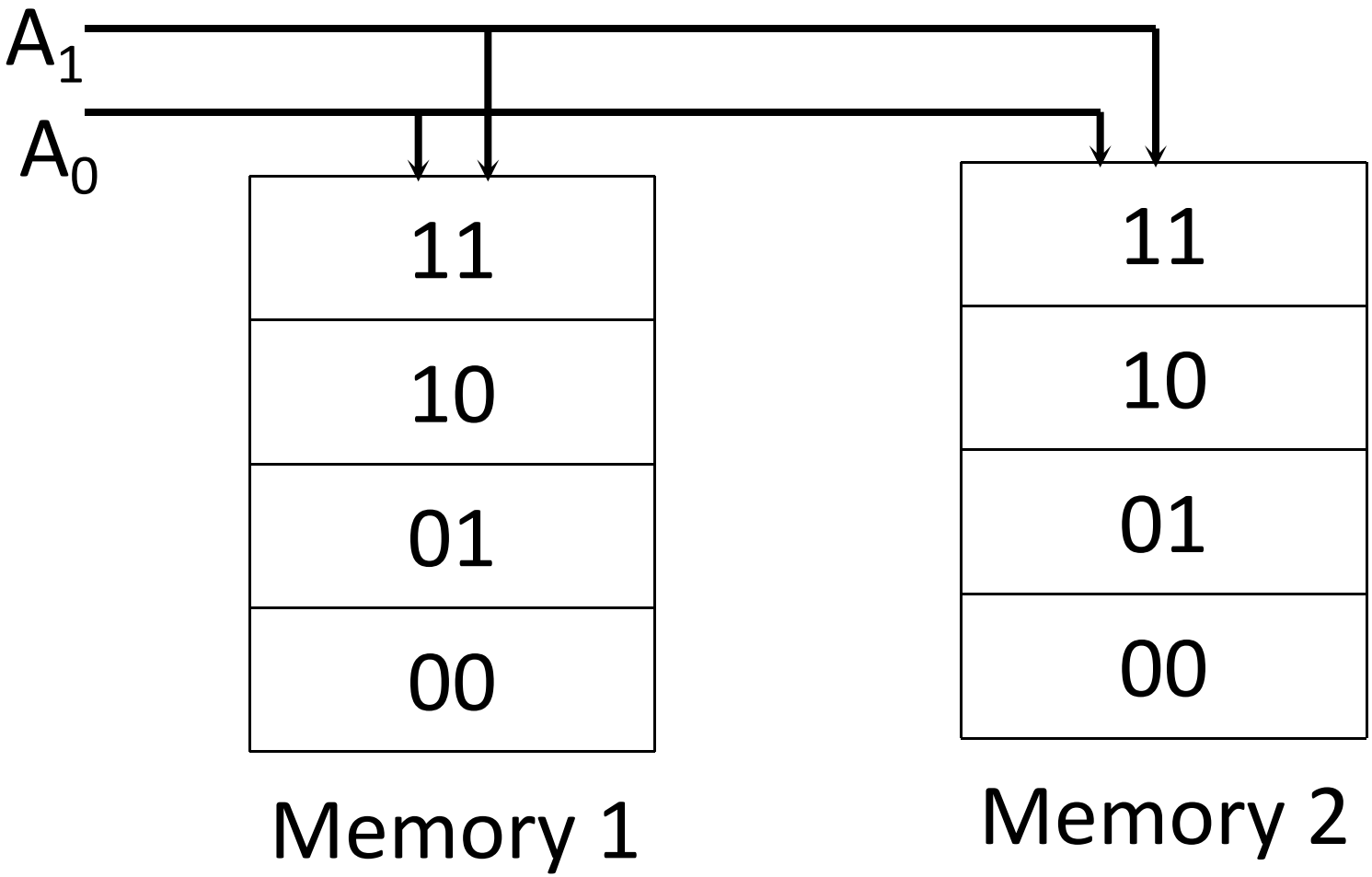
Memory Chip


'k' data input lines
'n' address lines
read
write
Memory
2n words
'k' bits per word
Chip select
'k' data output lines

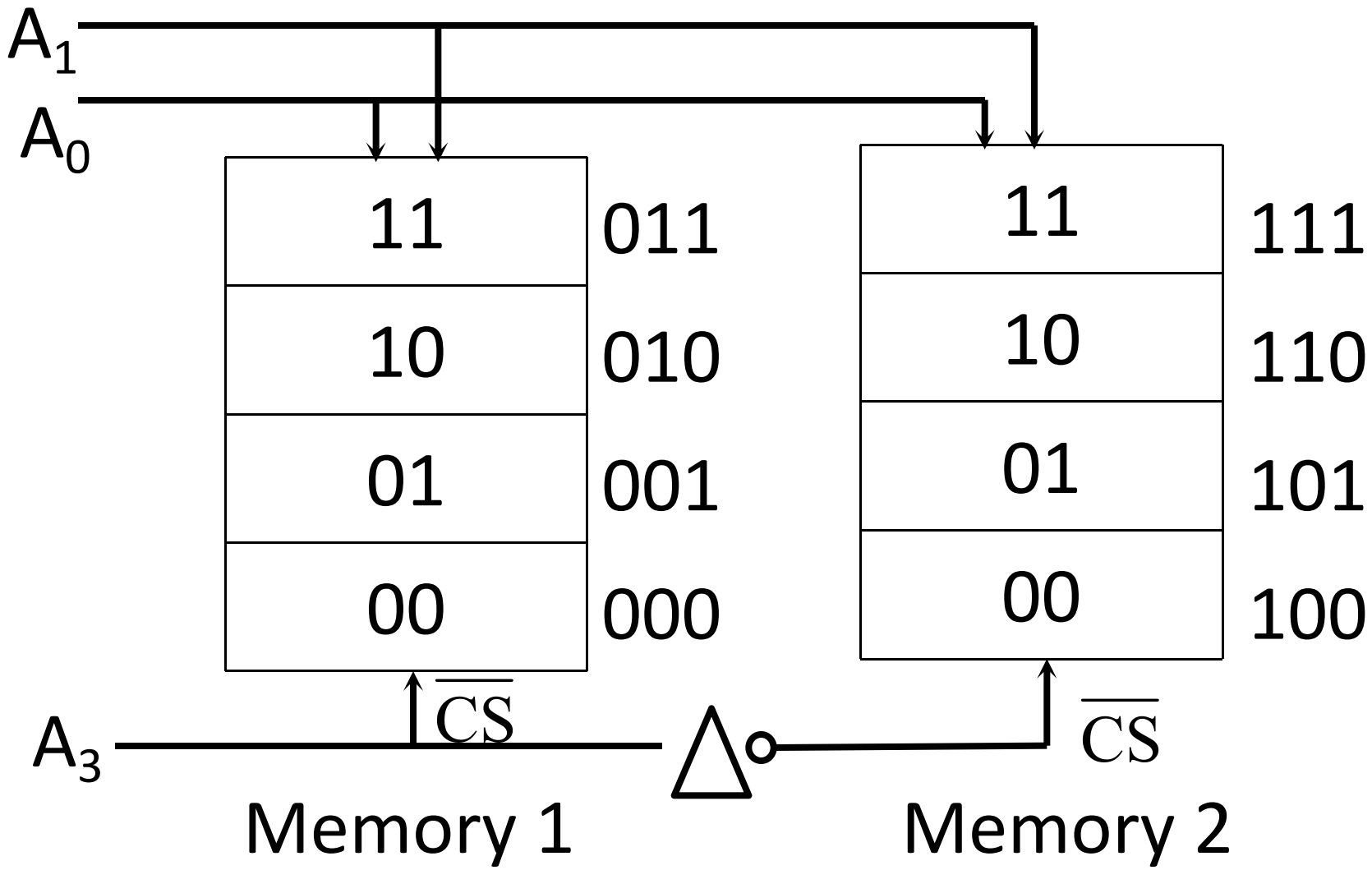
8085 Interfacing with Memory chips



Interface with two memory chips



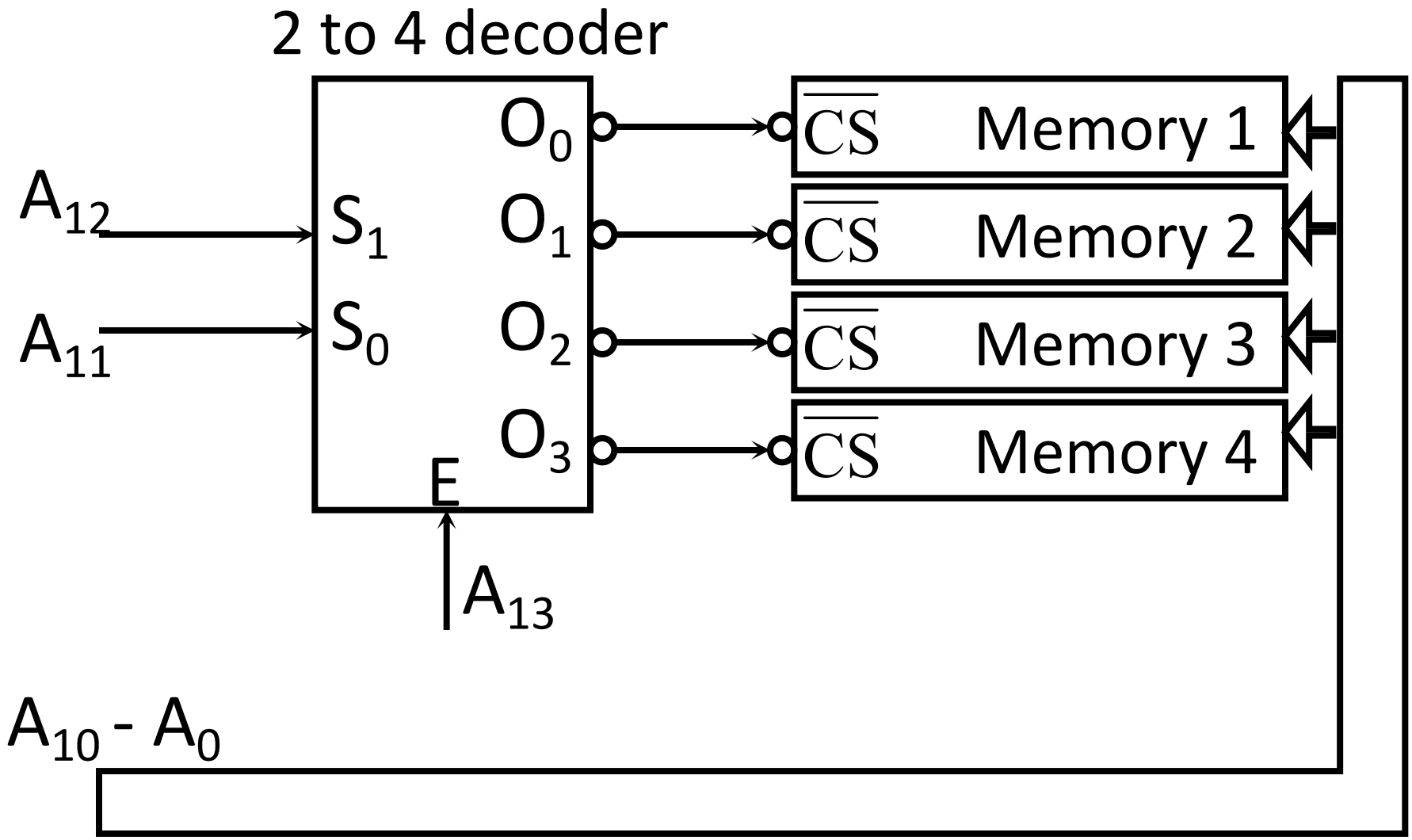
Interface with two memory chips



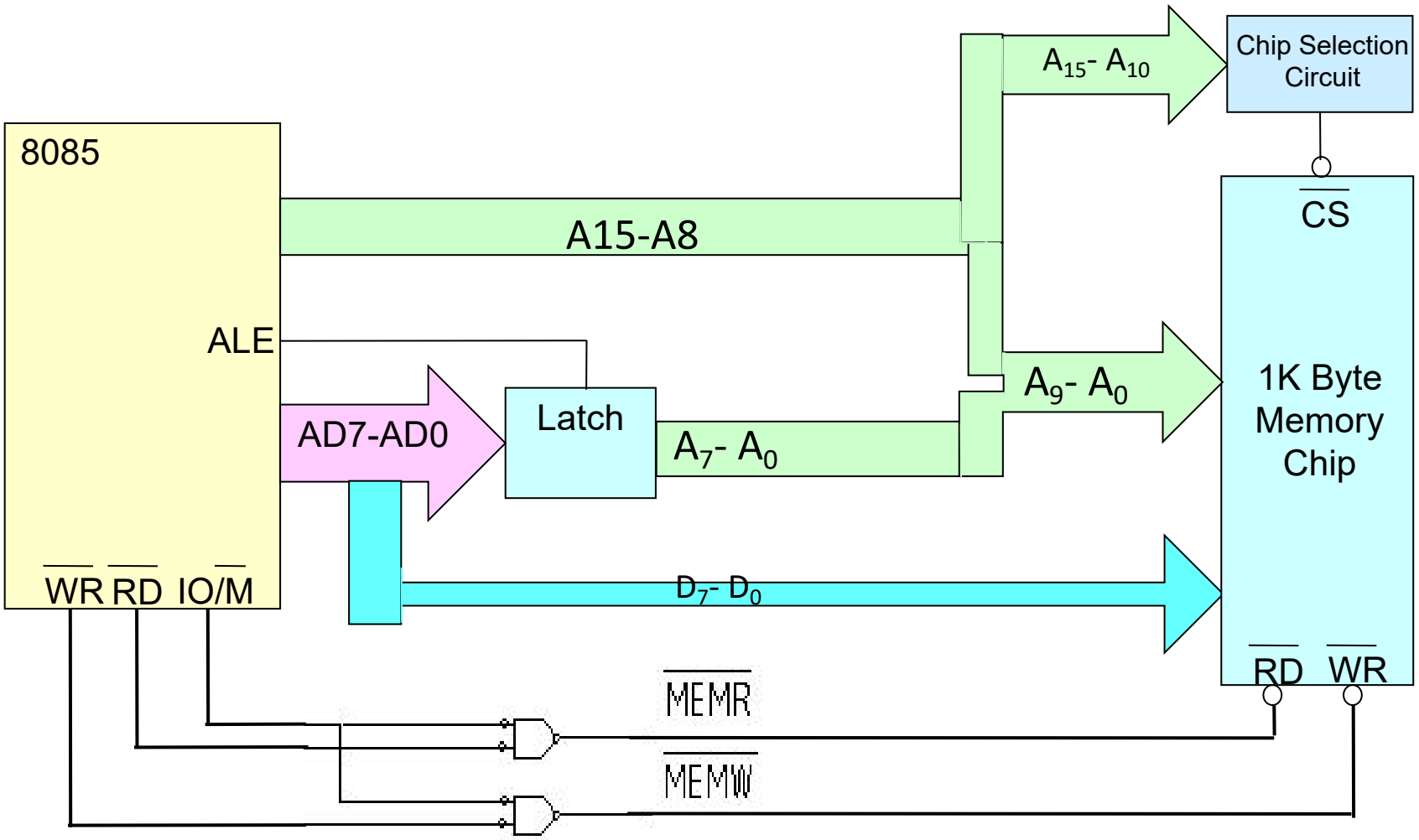
Interface with Multiple Chips

- In case of multiple chips simple circuit like NOT gate will not work.
- In this case normally decoder circuits like 3-to-8 decoder circuit 74LS138 are used.
- These circuit are called address decoders.

Address decoders



The Overall Picture

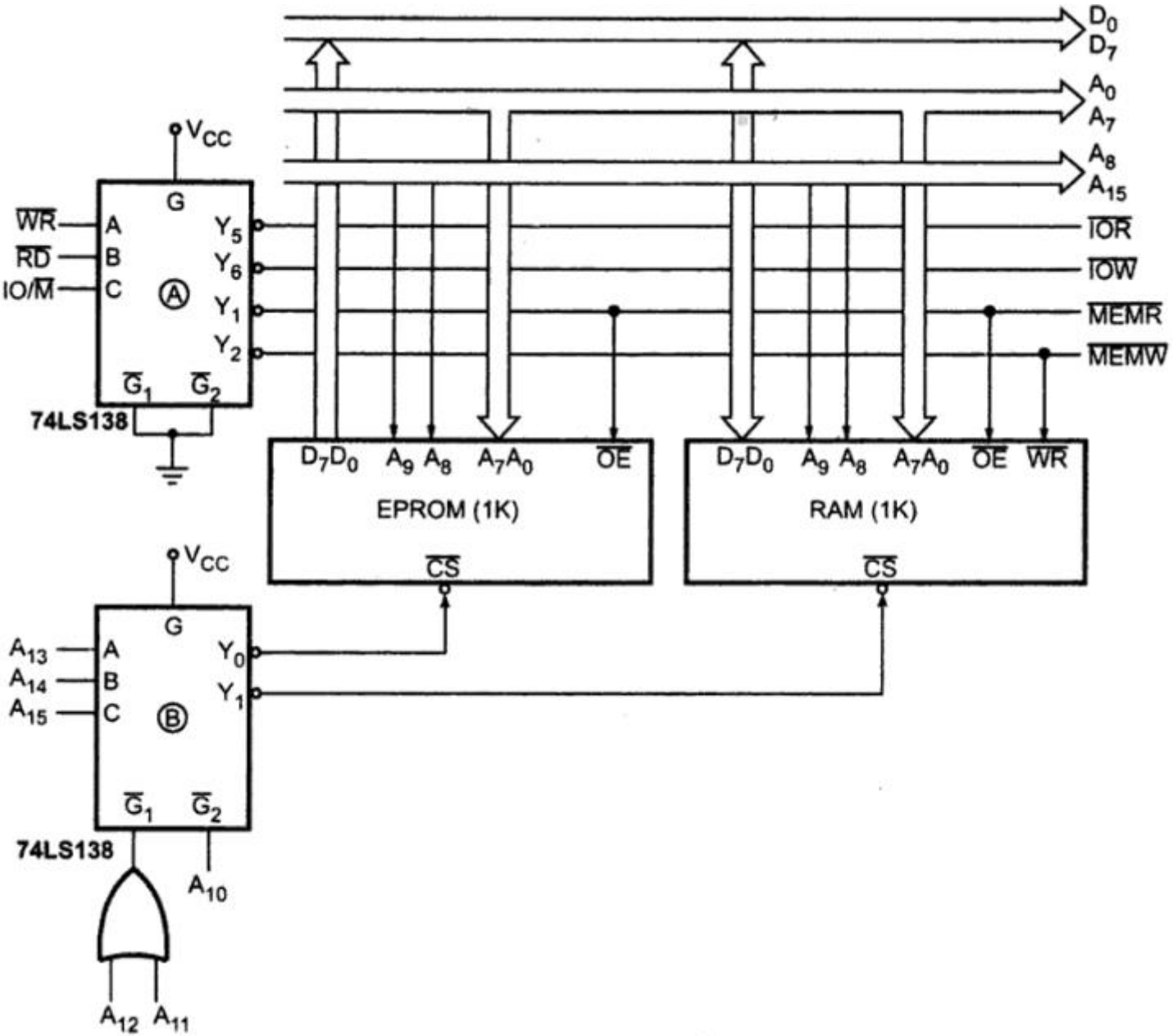


Types of Address Decoding

- There are two types of address decoding techniques
  - Exhaustive Decoding
  - Partial Decoding

Exhaustive Decoding

- In this type of scheme all the 16 bits of the 8085 address bus are used to select a particular location in memory chip.
- Advantages:
  - Complete Address Utilization
  - Ease in Future Expansion
  - No Bus Contention, as all addresses are unique.
- Disadvantages
  - Increased hardware and cost.
  - Speed is less due to increased delay.

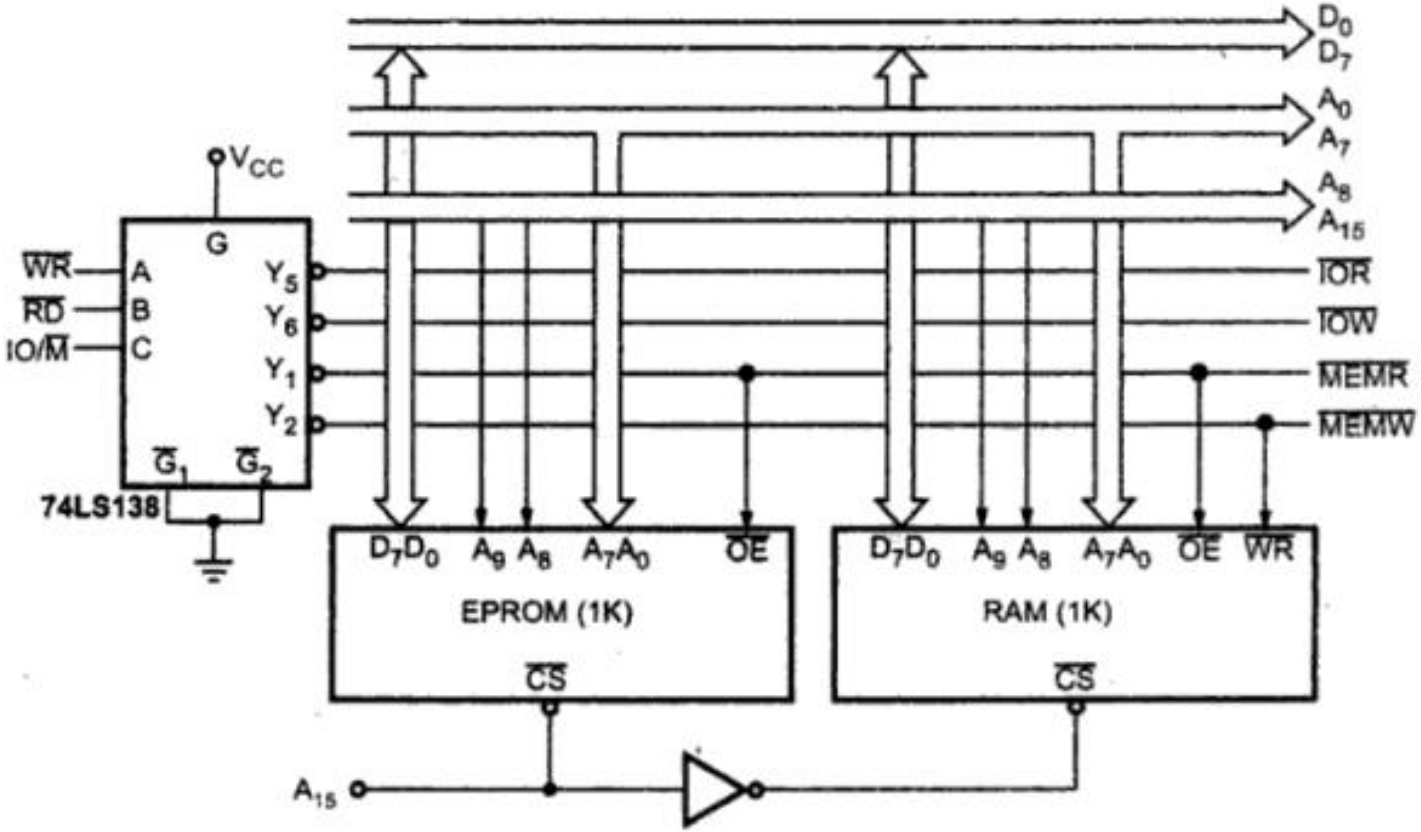

D0
D7
A0
A7
A8
A15
VCC
G
WR
RD
IO/M
A
B
C
Y5
Y6
Y1
Y2
G1
G2
74LS138
IOR
IOW
MEMR
MEMW
D7D0
A9
A8
A7A0
OE
WR
EPROM (1K)
RAM (1K)
CS
A13
A14
A15
Y0
Y1
A10
A12
A11

Memory Map :

Memory ICs	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Address
Starting address of EPROM	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H
End address of EPROM	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	03FFH
Starting address of RAM	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000H
End address of RAM	0	0	1	0	0	0	1	1	1	1	1	1	1	1	1	1	23FFH

Partial Decoding

- In this scheme minimum number of address lines are used as required to select a memory location in chip.
- Advantages:
  - Simple, Cheap and Fast.
- Disadvantages:
  - Unutilized space & fold back (multiple mapping).
  - Bus Contention.
  - Difficult future expansion.


D0
D7
A0
A7
A8
A15
VCC
G
WR
RD
IO/M
A
B
C
Y5
Y6
Y1
Y2
G1
G2
74LS138
IOR
IOW
MEMR
MEMW
D7D0
A9
A8
A7A0
OE
EPROM (1K)
CS
D7D0
A9
A8
A7A0
OE
WR
RAM (1K)
CS
A15

Memory Map :

Memory ICs	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Address
Starting address of EPROM	0	X	X	X	X	X	0	0	0	0	0	0	0	0	0	0	0000H
End address of EPROM	0	X	X	X	X	X	1	1	1	1	1	1	1	1	1	1	03FFH
Starting address of RAM	1	X	X	X	X	X	0	0	0	0	0	0	0	0	0	0	8000H
End address of RAM	1	X	X	X	X	X	1	1	1	1	1	1	1	1	1	1	83FFH

Full Address Decoding	Partial Address Decoding
1. All higher address lines are decoded to select the memory or I/O device.	1. Few higher address lines are decoded to select the memory or I/O device.
2. More hardware is required to design decoding logic.	2. Hardware required to design decoding logic is less and sometimes it can be eliminated.
3. Higher cost for decoding circuit.	3. Less cost for decoding circuit.
4. No multiple addresses.	4. It has a disadvantage of multiple addresses (shadow addresses).
5. Used in large systems.	5. Used in small systems.

- Design a microprocessor system for the 8085 microprocessor such that it should contain 2Kbyte of EPROM and 2KByte of RAM with starting addresses 0000H and 6000H respectively.

EXAMPLE

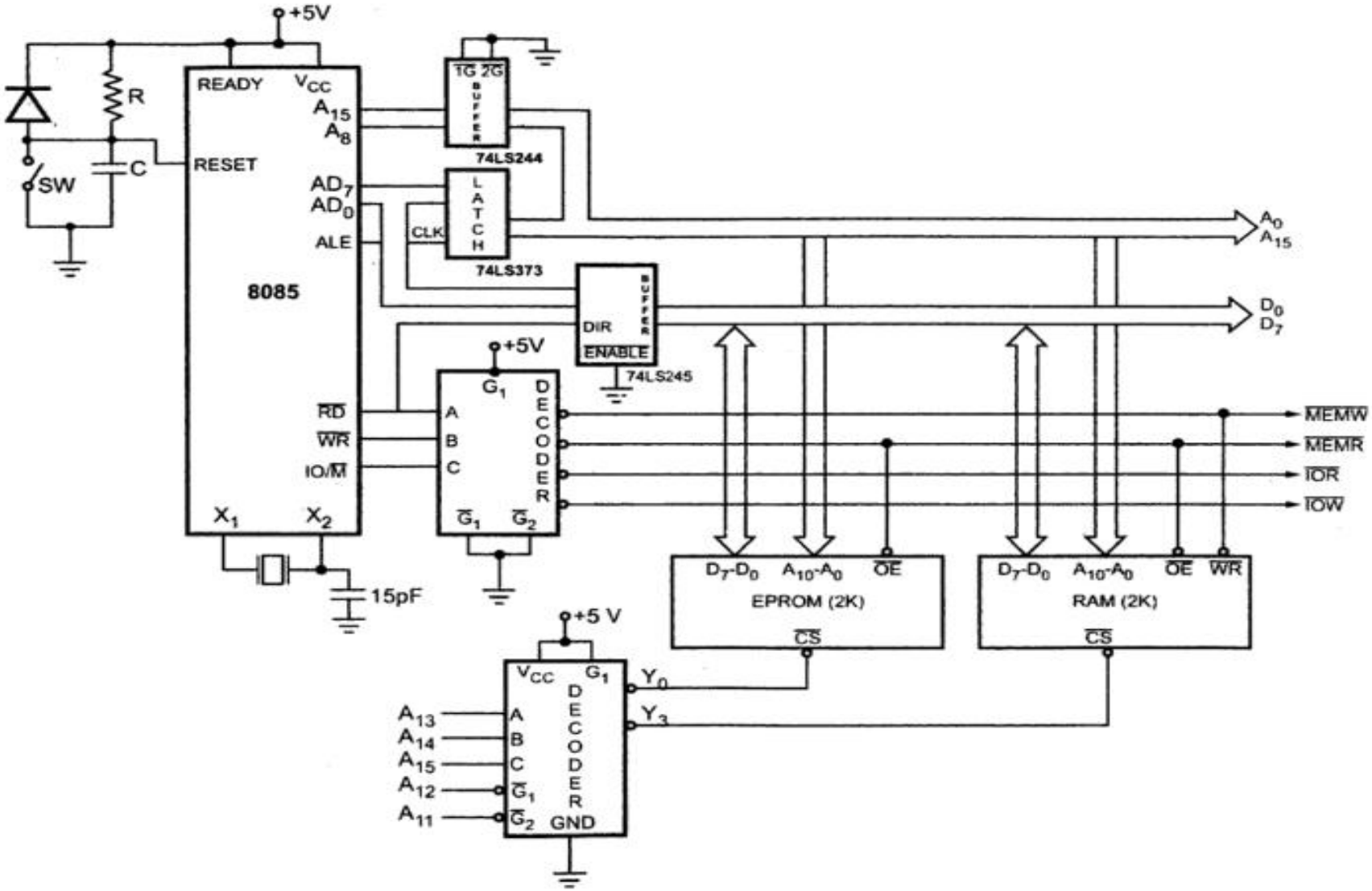


Fig. 6.6 Memory system using 2Kbyte EPROM and 2Kbyte RAM

EXAMPLE

Memory ICs	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Address
Starting address of EPROM	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H
End address of EPROM	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	07FFH
Starting address of RAM	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	6000H
End address of RAM	0	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	67FFH

Table 6.7 Memory map

Interfacing I/O Devices

- Using I/O devices data can be transferred between the microprocessor and the outside world.
- This can be done in groups of 8 bits using the entire data bus. This is called parallel I/O.
- The other method is serial I/O where one bit is transferred at a time using the SID and SOD pins on the Microprocessor.

Types of Parallel Interface

- There are two ways to interface 8085 with I/O devices in parallel data transfer mode:
  - Memory Mapped IO
  - IO Mapped IO

Memory Mapped IO

- It considers them like any other memory location.
  - They are assigned a 16-bit address within the address range of the 8085.
  - The exchange of data with these devices follows the transfer of data with memory. The user uses the same instructions used for memory.

IO Mapped IO

- It treats them separately from memory.
  - I/O devices are assigned a “port number” within the 8-bit address range of 00H to FFH.
  - The user in this case would access these devices using the IN and OUT instructions only.

IO mapped IO V/s Memory Mapped IO

Memory Mapped IO

- IO is treated as memory.
- 16-bit addressing.
- More Decoder Hardware.
- Can address 2^{16}=64k locations.
- Less memory is available.

IO Mapped IO

- IO is treated IO.
- 8- bit addressing.
- Less Decoder Hardware.
- Can address 2^{8}=256 locations.
- Whole memory address space is available.

IO mapped IO V/s Memory Mapped IO

Memory Mapped IO

- Memory Instructions are used.
- Memory control signals are used.
- Arithmetic and logic operations can be performed on data.
- Data transfer b/w register and IO.

IO Mapped IO

- Special Instructions are used like IN, OUT.
- Special control signals are used.
- Arithmetic and logic operations can not be performed on data.
- Data transfer b/w accumulator and IO.

The interfacing of output devices

- Output devices are usually slow.
- Also, the output is usually expected to continue appearing on the output device for a long period of time.
- Given that the data will only be present on the data lines for a very short period (microseconds), it has to be latched externally.

The interfacing of output devices

- To do this the external latch should be enabled when the port's address is present on the address bus, the IO/M signal is set high and WR is set low.
- The resulting signal would be active when the output device is being accessed by the microprocessor.
- Decoding the address bus (for memory-mapped devices) follows the same techniques discussed in interfacing memory.

Interfacing of Input Devices

- The basic concepts are similar to interfacing of output devices.
- The address lines are decoded to generate a signal that is active when the particular port is being accessed.
- An IORD signal is generated by combining the IO/M and the RD signals from the microprocessor.

Interfacing of Input Devices

- A tri-state buffer is used to connect the input device to the data bus.
- The control (Enable) for these buffers is connected to the result of combining the address signal and the signal IORD.

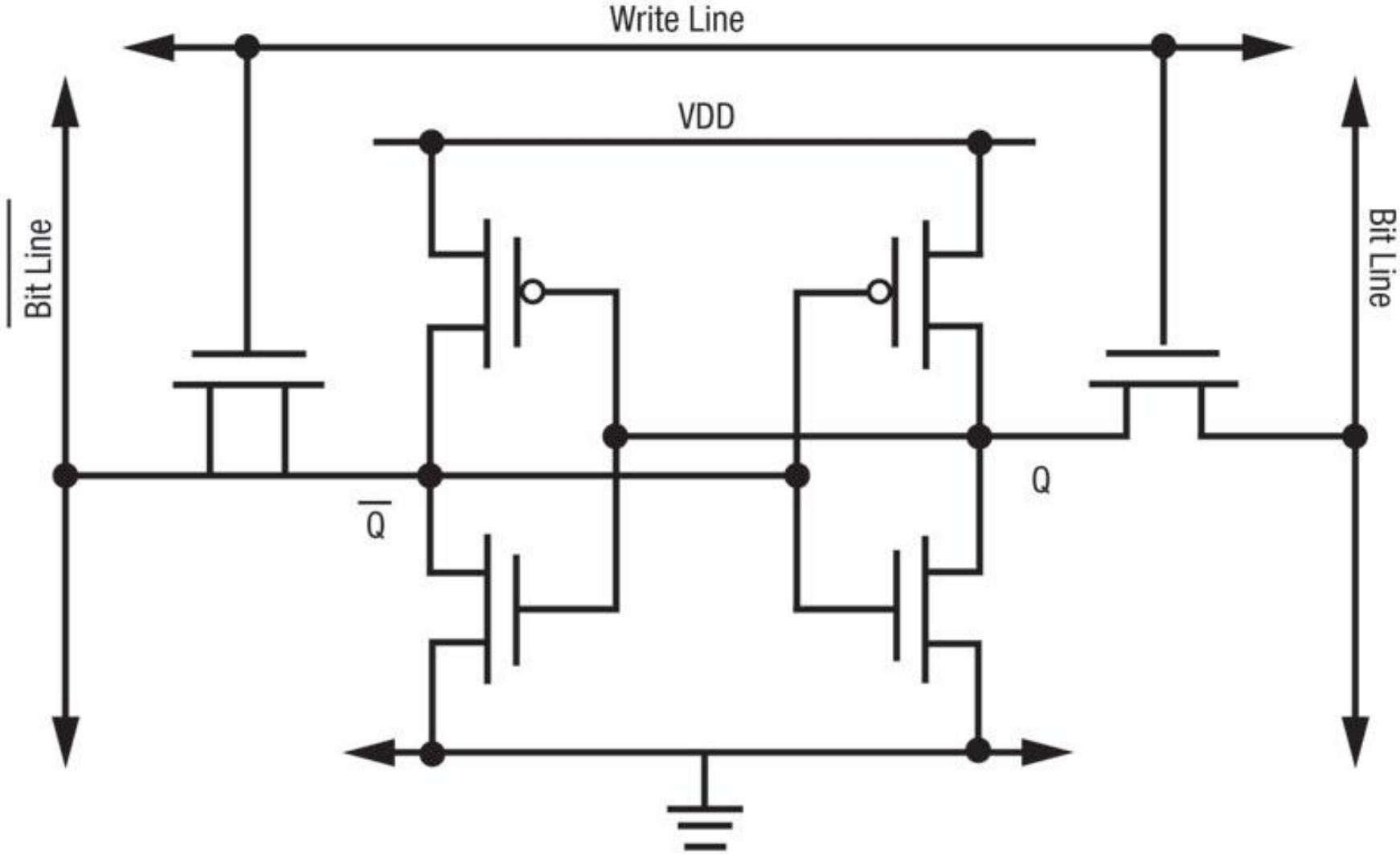
Basic RAM Cell

- RAM is a type of computer memory that can be accessed randomly i.e. any location can be accessed any time within chip.
- It is most common type of memory found in computers, printers etc.
- It is basically of two types:
  - SRAM
  - DRAM

SRAM

- SRAM stands for Static Random Access Memory.
- This memory is made up of flip-flops and stores the bit as a voltage.
- Each cell requires 6 transistors hence chip has low density but high speed.
- More expensive and consumes more power.
- Often known as cache memory in high speed PCs.

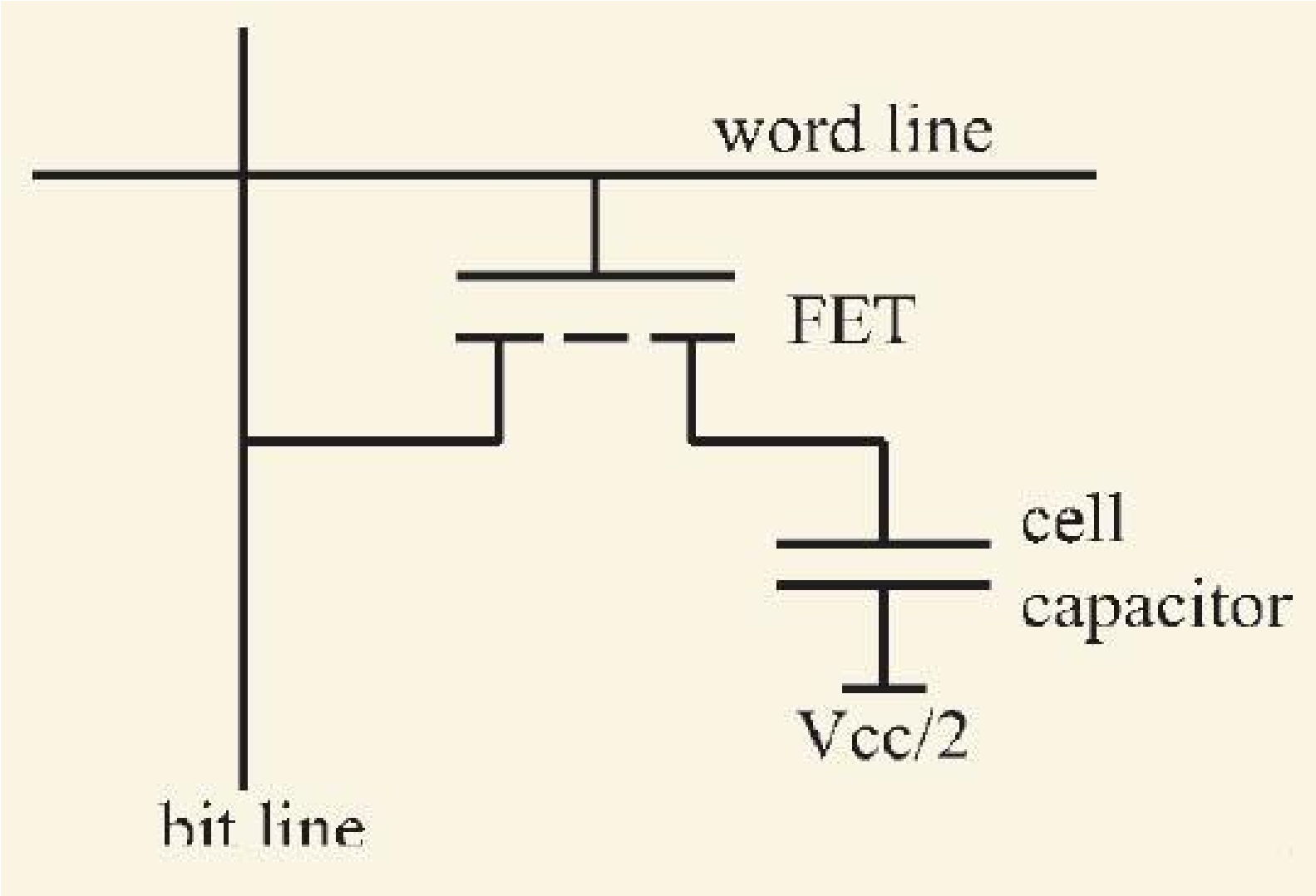
Basic SRAM Cell



DRAM

- DRAM stands for Dynamic Random Access Memory.
- This memory is made up of MOS transistor gates and it stores the bit as charge.
- High density, low power consumption, cheap as compared to SRAM.
- Due to leakage of charge requires frequent refreshing and hence extra circuitry.

Basic DRAM



ROM

- ROM is a read only memory.
- It retains the information even if power is turned off.
- It contains permanently stored instructions that help in staring up of a computer e.g. BIOS or Basic Input Output System.
- These are of following three basic types
  - PROM, EPROM, EEPROM

PROM

- The Programmable Read Only Memory can be programmed only once in its lifetime.
- Information once stored can not be erased.
- Requires special hardware circuit to program it.

EPROM

- Stands for Erasable Programmable Read Only Memory.
- These ROMs can be erased and programmed again and again.
- Can be erased with UV light or electricity.
- Main disadvantage is that it takes 15 to 20 minutes to erase it.

EEPROM

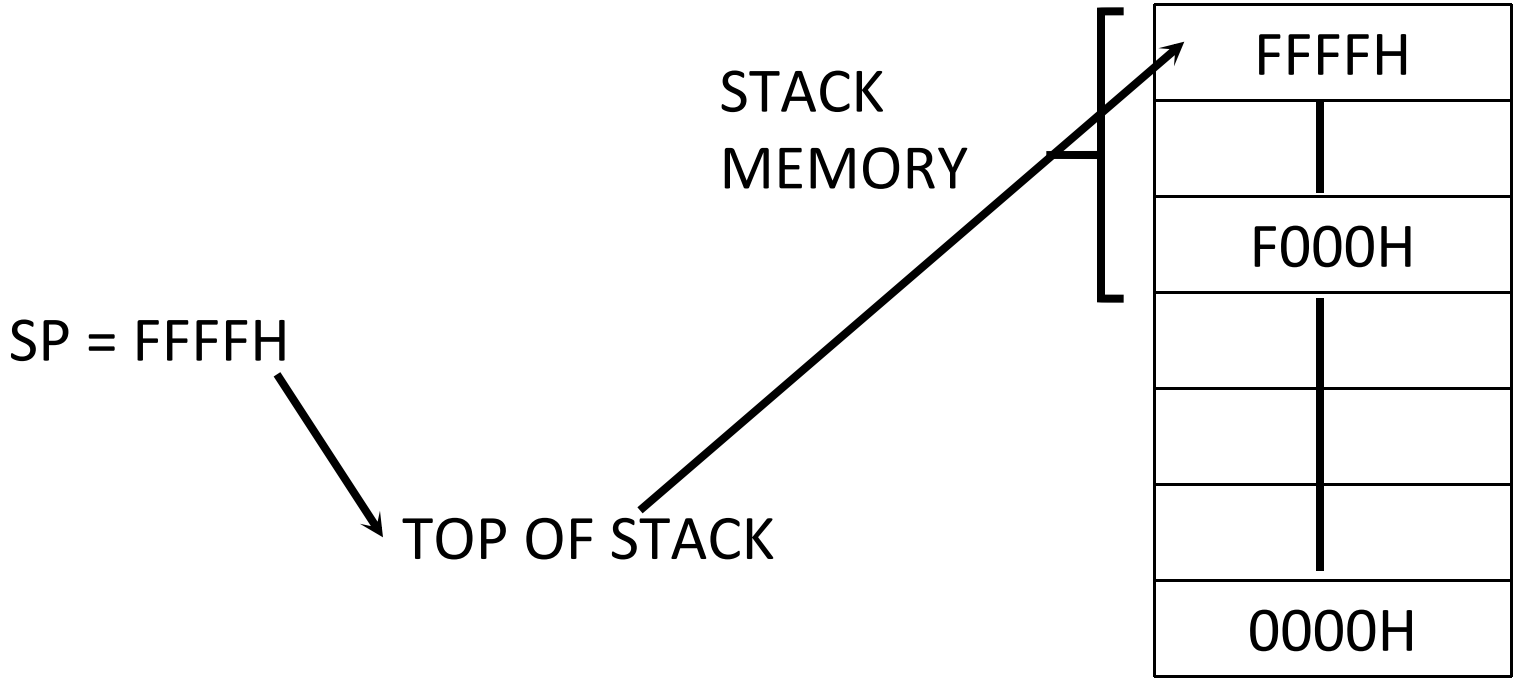
- Stands for Electrically Erasable Programmable Read Only Memory.
- Information can be erased electrically at register level rather than erasing entire information.
- It requires lesser erasing time.

Stack

- It is a part of memory, reserved in RAM, used to temporarily store information during execution of program.
- Starting address of stack is loaded in “Stack Pointer (SP)” (a 16-bit register).
- The address pointed to by SP is known as “Top of Stack”, which is always an empty memory location.

Stack Initialization

- Stack can be defined anywhere in RAM.
- But generally it initialized from highest (end) address of RAM to avoid any data loss.



Size of Stack Memory

- Theoretically there is no limitation on the size of stack memory.
- Practically the size of stack memory is limited to the availability of free RAM.
- As RAM is used to store temporarily program and data during execution, hence only free RAM can be used as stack.

Storing Data on Stack

- Stack is Last-In-First-Out (LIFO) type of memory.
- When information is stored on stack, the Stack Pointer register decrements to point to lower empty address.
- When information is read from stack, the Stack Pointer register increments to point to higher empty address.

Animation Stack Memory

PUSH B

POP B

PUSH C

POP C

STACK MEMORY

FFFF	52 H
FFFE	35 H
FFFD	
FFFC	
FFFB	
FFFA	
FFF9	
0001	
0000	

B= 52 H

C = 35 H

FFFD H

Stack Pointer

Advantages of Stack

- Address is always in Stack Pointer, need not be part of instruction, therefore, stack access is always faster.
- Stack instructions are short with only one operand.
- Used to save important data before branch instruction e.g. jump or interrupt instruction.